

Step 2 — NLP Preprocessing

Progress Report: Concepts, Methods & Justifications

Project: A Machine Learning Approach to Financial News
Sentiment and Market Trend Prediction

Student: Khelil Dhiaeddine

Supervisor: Nesrine Lahiani

Teacher: Hadjer Ykhlef

Target Stock: Tesla (TSLA) — January 2020 to December 2023

Dataset: ~87,196 documents (News, Reddit, Twitter General, Twitter Musk)

April 2026

Table of Contents

1. Introduction — The Step 2 Sub-Pipeline

2. Phase 1: FinBERT-First Diagnostic (Notebook 02)

- 2.1. Why Diagnose Before Cleaning
- 2.2. Stratified Sampling Strategy
- 2.3. Token Length Analysis
- 2.4. Noise Pattern Detection
- 2.5. FinBERT Inference on Raw Text
- 2.6. Diagnostic Findings

3. Phase 2: Source-Specific Cleaning (Notebook 03)

- 3.1. Why Source-Specific, Not Universal
- 3.2. Cleaning Functions per Source
- 3.3. Unit Testing
- 3.4. Before vs. After Validation with FinBERT
- 3.5. Full Dataset Cleaning & Quality Report

4. Phase 3: NLP Preprocessing Pipeline (Notebook 04)

- 4.1. Tokenization — Source-Aware Strategy
- 4.2. Stopword Removal — Financial Adjustments
- 4.3. Lemmatization — spaCy over NLTK
- 4.4. Named Entity Recognition (NER)
- 4.5. Final Export & Schema

5. Architectural Decisions Summary

6. Output & What Comes Next

1. Introduction — The Step 2 Sub-Pipeline

Step 2 of the project pipeline transforms the raw unified dataset (Layer 2, ~87,196 documents across four sources) into a representation suitable for sentiment analysis models. Rather than applying a single monolithic preprocessing script, the work was divided into three sequential notebooks, each with a distinct purpose and a clear deliverable.

Notebook	Purpose	Output
02_eda_finbert_pilot	Run FinBERT on raw text to diagnose what breaks before writing cleaning code	Cleaning checklist per source
03_nlp_cleaning_by_source	Source-specific noise removal, validated via before/after FinBERT comparison	layer2_cleaned.parquet
04_nlp_preprocessing	Tokenization, stopword removal, lemmatization, NER	layer2_preprocessed.parquet

Core design principle: Every preprocessing decision is driven by empirical evidence, not assumption. The FinBERT diagnostic notebook (02) was run first specifically to identify what cleaning was necessary per source before any code was written. This avoids the common mistake of blindly applying a generic preprocessing recipe that may help some sources while hurting others.

Two-track architecture: The preprocessed output serves two downstream consumers in Step 3. FinBERT (the primary model, Novelty 1) receives the cleaned raw text — it uses its own WordPiece tokenizer and should not receive lemmatized input. The classical baselines (TF-IDF + Logistic Regression, TF-IDF + SVM, VADER) receive the lemmatized token sequences. Both tracks are prepared in parallel within the same output file.

2. Phase 1: FinBERT-First Diagnostic (Notebook 02)

2.1. Why Diagnose Before Cleaning

The standard approach in most NLP projects is to write a cleaning pipeline first — removing URLs, mentions, special characters — and then feed the cleaned text to a model. This project inverts that order deliberately. By running FinBERT on completely unprocessed raw text first, we can observe exactly how the model responds to noise, which sources cause the most confusion, and which noise patterns are genuinely problematic versus harmless.

This approach has two concrete benefits. First, it produces an empirically grounded cleaning checklist rather than an assumed one. Second, it provides a baseline: we know what FinBERT outputs on raw text, so after cleaning, we can measure the exact improvement in confidence and label distribution. Without this baseline, any claim that 'cleaning improved model performance' would be unverifiable.

2.2. Stratified Sampling Strategy

Running FinBERT on all 87,196 documents for a diagnostic pass would be computationally wasteful. Instead, a stratified sample of 200 rows was drawn: exactly 50 rows per source (news, reddit, twitter_general, twitter_musk). This ensures equal representation across sources, which is critical because the diagnostic goal is to compare noise profiles between sources, not to measure aggregate statistics.

The sample was drawn with a fixed random seed (42) to ensure reproducibility. Null and empty texts were excluded before sampling to avoid wasting diagnostic slots on unusable rows.

2.3. Token Length Analysis

Before running inference, each text was tokenized using FinBERT's WordPiece tokenizer **without truncation** to measure the true token length. This is important because FinBERT has a hard limit of 512 tokens. Any text exceeding this length is silently truncated, meaning the model only sees the beginning of the document. For short tweets (typically 30–80 tokens), this is irrelevant. For news articles, which can reach 1,000+ tokens, truncation means the model may miss key information in the body of the article.

The analysis revealed that news articles frequently exceed the 512-token limit, while Twitter texts are well within bounds. Reddit posts fall in between. This finding directly informed the decision to keep headlines and content separate in the data architecture — a future experiment can compare headline-only versus full-content FinBERT performance.

2.4. Noise Pattern Detection

A set of regex-based detectors was applied to every row in the sample to flag specific noise patterns without removing them. The patterns scanned include: URLs, cashtags (\$TSLA), @mentions, hashtags, emojis, HTML artifacts, and excessive whitespace. The count of affected rows per pattern per source was recorded.

This scan is purely diagnostic — no text is modified. The output is a frequency matrix showing which noise patterns are prevalent in which sources. For example, twitter_musk texts were saturated with @mentions (nearly every row), while news texts had almost none. This evidence justifies building source-specific cleaning functions rather than a one-size-fits-all approach.

2.5. FinBERT Inference on Raw Text

FinBERT (ProsusAI/finbert) was loaded from HuggingFace and run in evaluation mode on the 200-row sample. The inference used first-chunk truncation at 512 tokens and batch processing (batch size 16) for efficiency. For each document, the model outputs three probability scores: P(positive), P(negative), and P(neutral). The predicted label is the one with the highest probability, and the confidence score is the maximum of the three probabilities.

2.6. Diagnostic Findings

The key findings that guided all subsequent cleaning decisions were:

- **High neutral dominance (70–84% per source):** On raw text, FinBERT predicted 'neutral' for the vast majority of documents. This does not mean the texts are genuinely neutral — it reflects the model's uncertainty when faced with noisy input. The noise confuses the model into defaulting to the safest class.
- **Low confidence scores:** Many predictions had a maximum probability below 0.60, indicating the model was essentially guessing between classes. These low-confidence predictions were concentrated in the noisiest sources (twitter_musk, twitter_general).
- **Source-specific noise profiles:** Each source has a distinct noise signature. Twitter Musk texts are heavy on @mentions and cashtags. Twitter General has dense URLs, emojis, and hashtags. Reddit has markdown artifacts and sarcastic language. News has minimal noise but truncation risk.
- **Auto-generated cleaning checklist:** Based on these findings, a per-source cleaning priority list was generated automatically. Any noise pattern appearing in more than 5 of the 50 sample rows for a given source was flagged as a cleaning priority for that source.

3. Phase 2: Source-Specific Cleaning (Notebook 03)

3.1. Why Source-Specific, Not Universal

A universal cleaning function (one function applied identically to all sources) would be simpler to implement but academically weaker and practically less effective. The FinBERT diagnostic showed that each source has a fundamentally different noise profile. Applying the same aggressive URL/mention stripping to news articles (which have almost no URLs) would be harmless but wasteful, while failing to strip markdown artifacts from Reddit would leave noise that confuses the model.

More importantly, some cleaning operations are source-dependent by nature. For example, cashtag normalization (\$TSLA → TSLA) preserves the financial signal token rather than stripping it entirely. This matters for Twitter where cashtags are the primary way users reference stocks. In news text, the same stock is referenced as 'Tesla' or 'TSLA' in prose, so cashtag handling is irrelevant.

3.2. Cleaning Functions per Source

News Cleaning

- Normalize excessive whitespace and newline characters to single spaces
- Preserve the full text content — news articles are well-structured prose with minimal noise
- No URL stripping needed (URLs are rare in GDELT-sourced article text)

Twitter General Cleaning

- Remove URLs (very frequent — most tweets contain at least one link)
- Normalize cashtags: \$TSLA → TSLA (preserve the ticker, remove the dollar sign)
- Remove @mentions (noisy for sentiment, the identity of the mentioned user is not relevant)
- Remove emojis (they confuse FinBERT's tokenizer and add no financial signal)
- Collapse repeated characters: l00000l → l000l
- Normalize whitespace

Twitter Musk Cleaning

- Same as Twitter General, with additional handling for the very high density of @mentions
- Musk's tweets tend to have heavy mention saturation — the cleaning must remove these without stripping legitimate content words

Reddit Cleaning

- Remove markdown artifacts (bold, italic, strikethrough, headers, links)
- Remove Reddit-specific boilerplate: 'Edit:', 'TL;DR:', 'Update:', '[deleted]', '[removed]'
- Normalize cashtags (same logic as Twitter)
- Remove URLs
- Normalize whitespace

3.3. Unit Testing

Every cleaning function was validated with explicit test cases before being applied to the dataset. For each source, a set of input/expected-output pairs was defined, covering edge cases such as: empty

strings, texts with only noise (should produce an empty string), texts with no noise (should pass through unchanged), and texts with mixed noise and content.

This is important for academic rigor — every transformation applied to the data should be verifiable and reproducible. The test cases also serve as documentation: a reader of the thesis can see exactly what each cleaning function does by looking at the test pairs.

3.4. Before vs. After Validation with FinBERT

This is the core validation step. After cleaning the 200-row sample, FinBERT was re-run on the same rows using the cleaned text. The comparison between the raw-text predictions (from notebook 02) and the clean-text predictions (from this step) measures the concrete impact of cleaning.

The metrics compared were:

- **Neutral prediction rate:** Did the proportion of neutral labels drop after cleaning? A drop indicates the model is now more decisive, which is the desired outcome.
- **Low-confidence rate:** Did the number of predictions below 0.60 confidence decrease?
- **Label shifts:** How many documents changed their predicted label after cleaning? Shifts from neutral to positive/negative are desirable; shifts between positive and negative would indicate cleaning introduced distortion (undesirable).

Documents flagged as 'too short' after cleaning (less than 3 tokens) were excluded from the FinBERT re-run to avoid noisy predictions on near-empty strings.

3.5. Full Dataset Cleaning & Quality Report

After validation on the 200-row sample, the same cleaning functions were applied to the full ~87,196 rows of Layer 2. The output file (`layer2_cleaned.parquet`) preserves all original Layer 2 columns and adds two new columns: `text_clean` (the cleaned text) and `is_too_short` (boolean flag for texts under 3 tokens after cleaning).

A quality report was generated summarizing: total rows, empty texts after cleaning, too-short texts, and usable rows per source. This report is included in the thesis data quality section and can also be used in the supervisor progress report.

4. Phase 3: NLP Preprocessing Pipeline (Notebook 04)

This notebook takes the cleaned dataset (`layer2_cleaned.parquet`) and applies four classical NLP preprocessing steps: tokenization, stopwords removal, lemmatization, and named entity recognition. Each step is source-aware — the logic adapts based on whether the document comes from news, Reddit, or Twitter.

Critical architectural note: FinBERT (the primary sentiment model) does **not** use the tokenized or lemmatized output. FinBERT operates on the cleaned raw text column directly, using its own internal WordPiece tokenizer. The preprocessing in this notebook is exclusively for the classical baseline models (TF-IDF + Logistic Regression, TF-IDF + SVM, VADER), which need tokenized and lemmatized input. Both tracks are prepared in parallel.

4.1. Tokenization — Source-Aware Strategy

What is Tokenization

Tokenization splits raw text into individual units (tokens) that downstream models can operate on. The choice of tokenizer directly affects the quality of all subsequent steps, because tokens are the atomic input to stopwords removal, lemmatization, and TF-IDF vectorization.

Why Two Tokenizers

Different text registers require different tokenizers. Social media text (Twitter) contains non-standard tokens like cashtags (\$TSLA), emoticons, and contractions (can't, won't). A standard tokenizer would split \$TSLA into two tokens (\$ + TSLA), losing the cashtag as a meaningful financial unit.

Source	Tokenizer	Why
twitter_general twitter_musk	TweetTokenizer (NLTK)	Built for social media — handles cashtags, emoticons, contractions, repeated characters
reddit news	word_tokenize (NLTK / Punkt)	Standard Penn Treebank tokenization for structured prose — consistent with spaCy

Configuration Details

- **preserve_case=False:** All tokens are lowercased uniformly
- **reduce_len=True:** Normalizes repeated characters (e.g., 'loooooo' → 'loool', max 3)
- **strip_handles=False:** @mentions are kept as tokens at this stage — they were already removed during cleaning where appropriate

4.2. Stopword Removal — Financial Adjustments

What are Stopwords

Stopwords are high-frequency words that carry little semantic content (e.g., 'the', 'is', 'at'). Removing them reduces vocabulary size and noise for bag-of-words models like TF-IDF. However, a generic stopwords list is not suitable for financial text — several words that are normally considered stopwords carry critical financial sentiment signal.

Financial Tokens Preserved (Removed from Stopword List)

The following tokens are part of the standard NLTK English stopwords list but were deliberately kept in the text because they carry financial sentiment meaning:

- **Negations:** 'not', 'no', 'nor' — these flip polarity entirely ('not bullish' ≠ 'bullish')
- **Directional words:** 'up', 'down' — core price movement language
- **Intensifiers:** 'more', 'less', 'very', 'most' — modify sentiment strength
- **Temporal/opposition:** 'against', 'before', 'after' — temporal context for events

Source-Specific Noise Words Added

Additional tokens were added to the stopwords list per source because they appear very frequently but carry zero financial signal:

Source	Added Stopwords	Reason
Twitter	rt, via, amp, http, https, co	Retweet markers, link artifacts
Reddit	edit, tldr, update, removed, deleted, mod	Reddit UI boilerplate, not content
News	said, says, according, read, click, subscribe	Journalistic filler, CTAs

4.3. Lemmatization — spaCy over NLTK

What is Lemmatization

Lemmatization maps each token to its dictionary base form (lemma) using morphological analysis. For example: 'bought' → 'buy', 'falling' → 'fall', 'batteries' → 'battery'. This normalization is essential for TF-IDF-based models because without it, 'buy', 'bought', 'buying', and 'buys' would be treated as four unrelated terms, fragmenting the feature space.

Why spaCy, Not NLTK WordNetLemmatizer

NLTK's WordNetLemmatizer requires the user to manually specify the part-of-speech (POS) tag for each token. Without the correct POS tag, the lemmatizer defaults to treating everything as a noun, which means verbs are not lemmatized: 'bought' stays 'bought' instead of becoming 'buy'. spaCy's pipeline runs POS tagging automatically before lemmatization, so 'bought' is correctly identified as a past-tense verb and mapped to 'buy'. This produces significantly better results with no manual intervention.

Protected Terms

Certain financial domain terms are protected from lemmatization to preserve their meaning. For example: 'tesla', 'tsla', 'musk', 'elon', 'bullish', 'bearish', 'ipo', 'etf', 'sec'. These terms are already in their canonical form and lemmatizing them could produce incorrect results (e.g., spaCy might try to lemmatize 'bullish' to 'bull', losing the financial connotation).

Batch Processing

For efficiency on the full 87K-row dataset, lemmatization was performed using spaCy's `nlp.pipe()` method in batch mode rather than row-by-row `apply()`. This leverages spaCy's internal batching and is significantly faster for large datasets.

4.4. Named Entity Recognition (NER)

What is NER

Named Entity Recognition identifies and classifies named entities in text — persons, organizations, locations, monetary amounts, dates, etc. In this project, NER serves two specific purposes that are directly relevant to the thesis.

Purpose 1: TSLA Relevance Filtering

Not every document in the dataset necessarily references Tesla or TSLA directly. Some may have been collected through keyword matching that captured tangential mentions. NER provides a systematic way to verify that each document contains a genuine Tesla/TSLA reference. Documents without any detected Tesla entity are flagged (not deleted) — they can be excluded from sentiment aggregation in Step 4 to avoid introducing noise into the market prediction features.

Purpose 2: Musk Effect Tagging

The 'Musk Effect' is one of the core academic claims of this thesis — that Elon Musk's public statements cause measurable price movements in TSLA. NER tags every document that mentions 'Elon Musk' as a PERSON entity, enabling downstream analysis to separately measure the sentiment impact of Musk-mentioning documents versus general TSLA discussion.

Source-Specific NER Strategy

Source	Method	Justification
Twitter (both)	Regex rules only	\$TSLA cashtags are deterministic — no ML model overhead needed
Reddit	spaCy NER + regex rules	Text-based mentions ('Tesla', 'Musk') need NER; regex catches cashtags
News	spaCy NER (full model)	Journalistic text has rich entity references; spaCy performs well here

The spaCy model used is `en_core_web_sm` (small English model) which uses the OntoNotes 5 label set. Only entity labels relevant to financial analysis are retained: `ORG` (organizations), `PERSON` (named individuals), `MONEY` (monetary values), `DATE/TIME`, and `GPE` (geopolitical entities such as countries or cities).

Output Schema

Each document receives two new columns: `entities` (a JSON-serializable list of `{text, label}` dictionaries for each entity found) and `has_tsla_entity` (a boolean flag: True if Tesla, TSLA, or Musk appears as a detected entity).

4.5. Final Export & Schema

After all four preprocessing steps, the output is saved as `layer2_preprocessed.parquet`. The schema preserves all original Layer 2 columns and appends the new NLP columns:

Column	Type	Description
<code>doc_id</code>	<code>string</code>	Unique document identifier from Layer 2

Column	Type	Description
published_at	datetime	UTC-normalized publication timestamp
source	string	news / reddit / twitter_general / twitter_musk
text	string	Cleaned raw text (input for FinBERT)
ticker	string	Always TSLA
url	string	Source URL for deduplication
tokens_raw	JSON string	Raw token list after tokenization
tokens_filtered	JSON string	Tokens after stopword removal
tokens_lemma	JSON string	Lemmatized tokens (for TF-IDF baselines)
entities	JSON string	Detected named entities [{text, label}]
has_tsla_entity	boolean	True if Tesla/TSLA/Musk entity detected

List and dictionary columns are serialized to JSON strings for maximum cross-environment compatibility (Kaggle, local Python, GitHub CI). Downstream notebooks deserialize them on load.

5. Architectural Decisions Summary

The following table consolidates the key design decisions made during Step 2 and their justifications. Each decision is traceable to empirical evidence from the diagnostic notebook or to established NLP best practices.

Decision	Justification
Diagnose before cleaning (model-first approach)	Produces an evidence-based cleaning checklist. Avoids blind preprocessing.
Source-specific cleaning functions	Each source has a different noise profile (shown by the diagnostic). Universal cleaning under-cleans some sources and over-cleans others.
Cashtag normalization, not stripping	\$TSLA → TSLA preserves the financial signal token instead of losing it.
Two tokenizers (TweetTokenizer + Punkt)	Social media text requires specialized tokenization. Standard tokenizer breaks cashtags and emoticons.
Financial terms kept in stopword list	Negations (not, no) and directional words (up, down) flip or define sentiment polarity.
spaCy over NLTK for lemmatization	Automatic POS tagging produces correct lemmas without manual intervention.
Protected lemma terms	Financial domain terms (tesla, bullish, bearish) must not be further reduced.
Source-aware NER strategy	Regex for deterministic Twitter cashtags; spaCy NER for richer Reddit/news text.
FinBERT receives cleaned raw text, not lemmatized tokens	FinBERT uses WordPiece tokenization internally. Lemmatizing first distorts its pretrained vocabulary.
JSON serialization for list/dict columns	Ensures Parquet compatibility across environments (Kaggle, local, CI/CD).

6. Output & What Comes Next

Step 2 is now complete. The final output, `layer2_preprocessed.parquet`, contains all ~87,196 documents with both cleaned raw text (for FinBERT) and lemmatized tokens (for classical baselines). Named entity tags and TSLA relevance flags are included for downstream filtering.

Deliverables produced in Step 2:

- `day1_finbert_sample_results.parquet` — FinBERT predictions on 200 raw rows (diagnostic baseline)
- `layer2_cleaned.parquet` — Full dataset after source-specific cleaning (~87K rows)
- `layer2_preprocessed.parquet` — Final preprocessed dataset with tokens, lemmas, and entities

Next steps — Step 3: Sentiment Analysis

Track	Input Column	Model	Purpose
FinBERT (Novelty 1)	text (cleaned raw)	ProsusAI/finbert	Primary sentiment classifier. Project's core contribution.
Classical baselines	tokens_lemma → TF-IDF	Logistic Regression, SVM, VADER	Academic comparison baselines to demonstrate FinBERT's advantage.

Both tracks will produce per-document sentiment labels (POSITIVE / NEGATIVE / NEUTRAL) and probability scores. These will then be aggregated into daily sentiment indices in Step 4 (Feature Engineering), which serve as input features for the market prediction models in Step 5.

Thesis limitation to document:

Documents with `has_tsla_entity = False` will be reviewed before Step 3. Those that genuinely contain no Tesla reference will be excluded from sentiment aggregation to avoid introducing noise into the market prediction features. The count and source distribution of these excluded documents should be reported in the data quality section of the thesis.